AI with Rav

Learn AI the way you actually think.

DAY 12 of 30

MACHINE LEARNING

Naive Bayes — the postman who sorts by words

WHAT YOU'LL LEARN TODAY

- How the spam filter really works
- Training = just counting words
- Multiply the clues to get a chance
- The one "naive" trick that makes it fast
- The Bayes formula in plain words

Naive Bayes — the postman who sorts by words

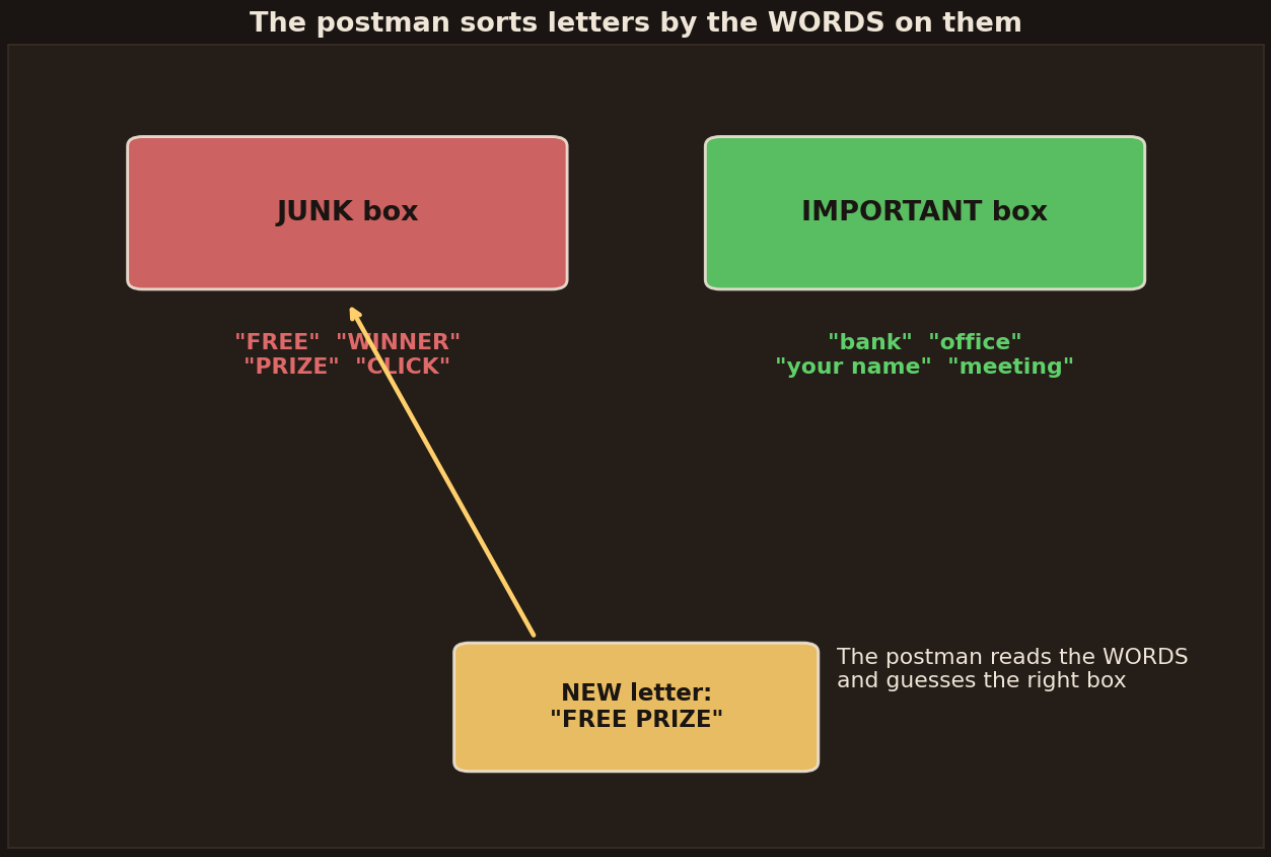
> In One Line: Imagine a postman who sorts letters into "Junk" and "Important" just by reading the WORDS on them. He learned from years of old letters which words show up where. Naive Bayes is that postman — turned into a super-fast machine.

Start here: the postman and two boxes

Picture an old postman with two boxes in front of him — a **JUNK box** and an **IMPORTANT box**. A new letter arrives. He doesn't open it or think hard. He just glances at the **words** on it. "FREE PRIZE" → junk. "meeting with your bank" → important. Done in a second.

How does he know? **Years of experience.** He has seen thousands of letters, and he remembers which words usually landed in which box. That's exactly how **Naive Bayes** works — the algorithm behind your Gmail spam filter.

The postman sorts by the words

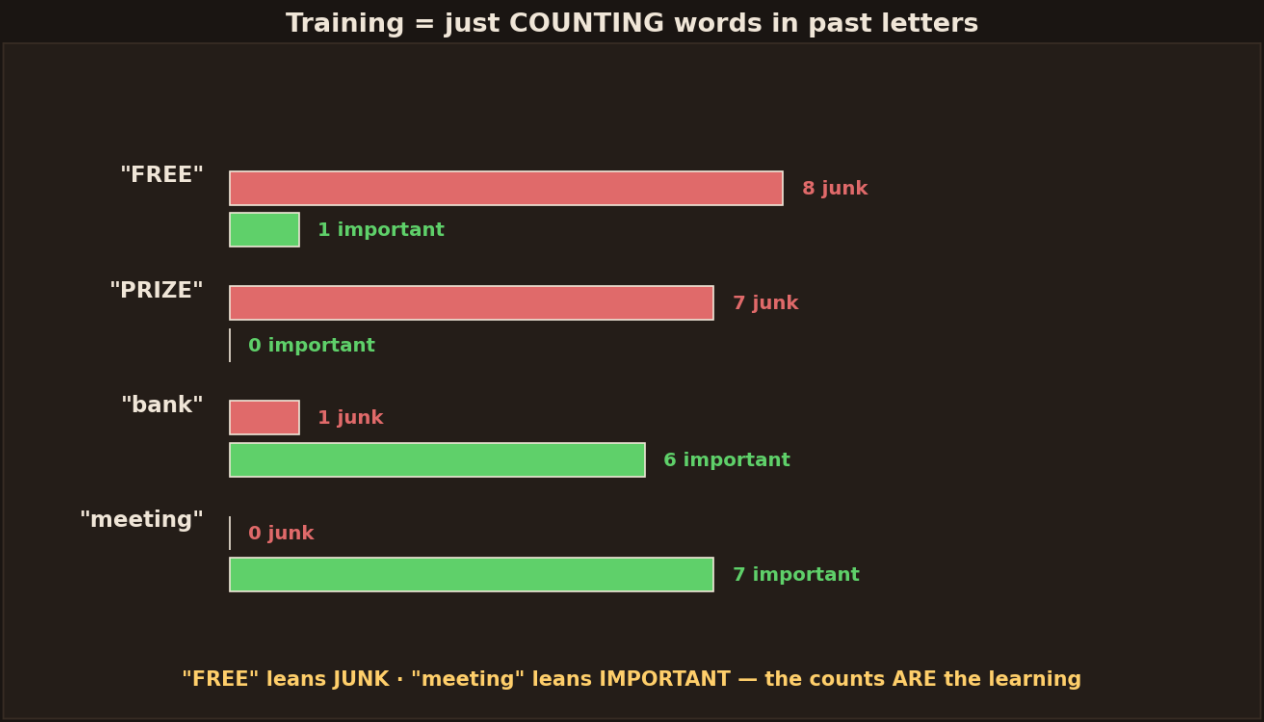


Two boxes — Junk and Important. Each has words that usually land in it. A new letter "FREE PRIZE" arrives, and the postman reads the words to guess the right box.

- Words like **"FREE", "WINNER", "PRIZE", "CLICK"** → usually **junk**.
- Words like **"bank", "office", "your name", "meeting"** → usually **important**.
- A new letter → read its words → guess the box those words point to.

> That's the whole idea in one line: **judge a letter by the words it carries.** No deep thinking, no opening the envelope. Just: "which box do these words usually belong to?" Simple — and shockingly good.

Training = just counting words



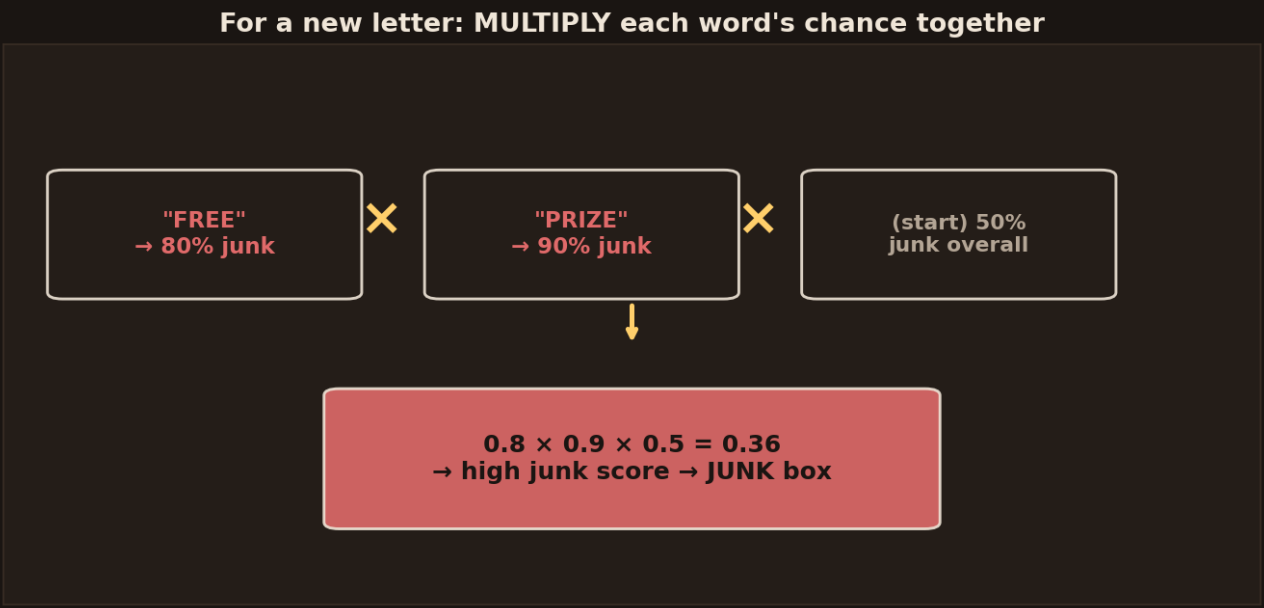
For each word, count how many times it appeared in Junk letters vs Important letters. "FREE" = 8 junk, 1 important. "meeting" = 0 junk, 7 important. The counts ARE the learning.

How does the machine "learn" like the postman? It does something almost silly-simple: it **counts.** It goes through thousands of old letters and, for each word, tallies how often it showed up in junk vs important.

- **"FREE"** → seen 8 times in junk, 1 in important. → leans **junk.**
- **"meeting"** → seen 0 times in junk, 7 in important. → leans **important.**

> That's the "training." No fancy maths, no rolling downhill like other models. Naive Bayes just **counts words** in the letters it has seen. This is why it's one of the **fastest** algorithms alive — training is basically counting.

For a new letter — multiply the clues



Each word gives a chance of junk. Multiply them together with the starting chance: 0.8 × 0.9 × 0.5 = 0.36. A high junk score → the letter goes in the Junk box.

Now a new letter comes: **"FREE PRIZE".** The machine looks up each word's junk-chance and **multiplies** them together:

- **"FREE"** → 80% junk (0.8).
- **"PRIZE"** → 90% junk (0.9).
- Start with how common junk is overall → say 50% (0.5).
- Multiply: **0.8 × 0.9 × 0.5 = 0.36** → a strong junk score → **JUNK box.**

> Why multiply? Because each word is a **clue**, and stacking clues makes you more sure. Two junky words together are *more* junky than one. Multiplying the chances is how the machine stacks the clues into one final score.

The one "naive" trick

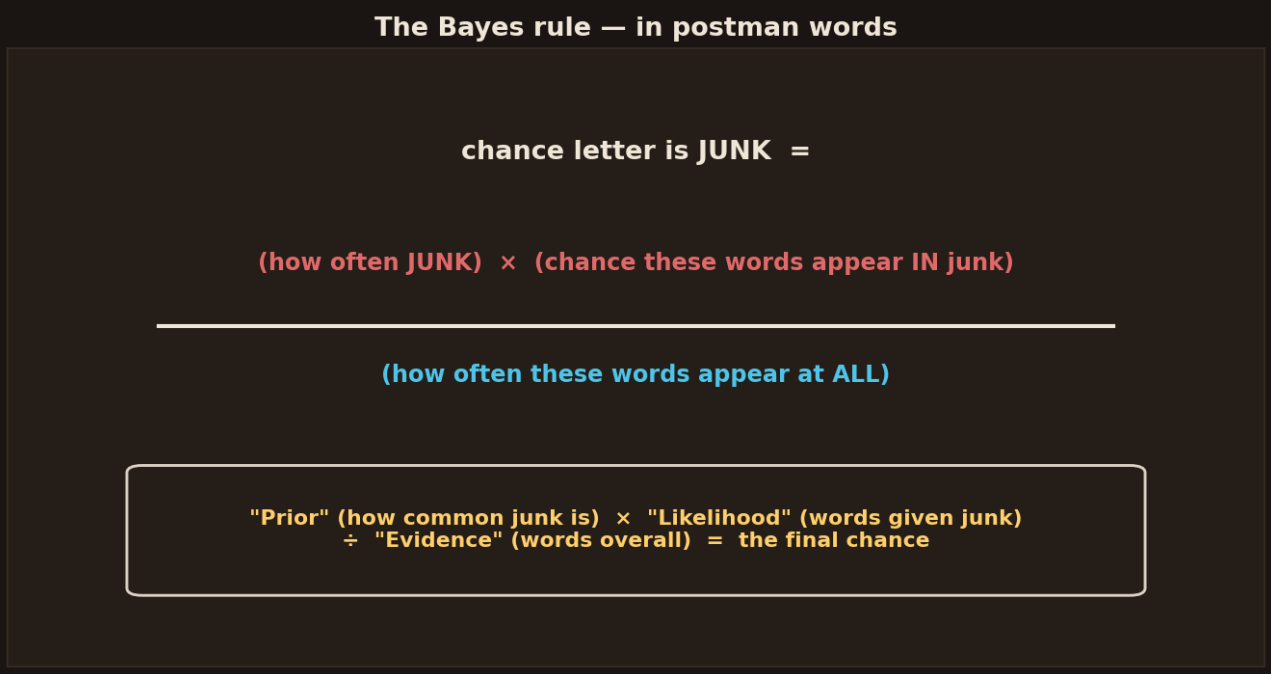
Left: in real language "New" and "York" go together — words depend on each other. Right: Naive Bayes PRETENDS every word stands alone. A wrong assumption — but it makes the maths lightning-fast, and it still works.

Here's where the funny name comes from. In real language, words **go together** — "New York", "credit card", "happy birthday". The word "York" almost always follows "New".

- **The truth:** words depend on each other.
- **What Naive Bayes pretends:** every word stands **completely alone**, like separate islands — it ignores that they go together.

> That assumption is **wrong** — words really do depend on each other. That's why it's called **"naive"** (a bit too innocent). But here's the surprise: pretending words are independent makes the maths **super fast and simple**, and for spam it *still works great.* A wrong shortcut that wins anyway — that's the magic of Naive Bayes.

The Bayes formula — in postman words



The Bayes rule as a simple fraction: (how common junk is) × (chance these words appear in junk), divided by (how often these words appear at all).

There's a real formula behind this — Bayes' rule — but in postman words it's just a fraction:

> **Chance a letter is JUNK = (how common junk is) × (chance these words show up in junk) ÷ (how often these words show up at all)**

Three simple pieces, each with a name:

- **Prior** → how common junk is *before* you read any words (the starting 50%).
- **Likelihood** → the chance these exact words appear *given* it's junk (from your counts).
- **Evidence** → how often these words appear overall (the bottom of the fraction — it just keeps the answer between 0 and 1).

> That's the entire formula: **start with a guess (prior), update it with the word-clues (likelihood), and keep it fair (evidence).** You don't need the symbols — you need this: it multiplies clues onto a starting guess. That's Bayes.

Train one — a few lines

```
from sklearn.naive_bayes import MultinomialNB
from sklearn.feature_extraction.text import CountVectorizer

# turn each email into word-counts
vec = CountVectorizer()
X = vec.fit_transform(emails)      # rows = emails, columns = word counts
y = labels                         # 1 = spam, 0 = not spam

model = MultinomialNB()
model.fit(X, y)                    # "fit" = just count the words!

print(model.predict(vec.transform(["FREE PRIZE claim now"])))  # -> [1] spam
```

> `CountVectorizer` does the word-counting; `MultinomialNB` is the postman. Notice `fit` is near-instant — it's only counting. This is why Naive Bayes is the classic first choice for **spam, news topics, and sentiment** (text problems).

Where it shines and where it slips

- **Super fast + tiny** → trains in a blink, works on huge text with little memory. Great baseline for any text task.
- **Loves text** → spam, topic-tagging, positive/negative reviews — words as clues is exactly its game.
- **The naive slip** → because it ignores word-order and links, it misses meaning like "not good" (it just sees "not" and "good" separately). For subtle language, deep learning wins.
- **A zero can break it** → if a word was never seen in junk, its chance is 0, and multiplying by 0 kills the whole score. The fix is a tiny trick called **smoothing** (add 1 to every count so nothing is ever truly zero).

> The one thing to remember: Naive Bayes is **fast, simple, and a fantastic starting point for text** — but its "every word alone" assumption means it doesn't truly *understand* language. Use it as your quick, strong baseline; reach for deeper models when meaning and word-order matter.

Recap — the 20-second version

- Naive Bayes = a **postman sorting letters by the words** on them.
- **Training is just counting** which words show up in each box — that's why it's lightning-fast.
- For a new letter, it **multiplies each word's chance** together into one score.
- The **"naive" trick**: pretend every word stands alone. Wrong, but fast — and it works.

- The formula = **prior × likelihood ÷ evidence** — a starting guess, updated by word-clues.

> Next up — Video 13: K-Means Clustering. So far every algorithm learned from labelled answers (spam / not-spam). But what if you have NO labels — just a pile of data — and you want the machine to find the natural groups by itself? See you Day 13.